FCFS

Program:

```
#include <stdio.h>

int main() {
  int n, curr_time = 0, pid[10], at[10], bt[10], ct[10], tat[10], wt[10],
        st[10], rt[10], i, j, temp;
  double tat_sum = 0, wt_sum = 0;

  printf("Enter no. of processes: ");
  scanf("%d", &n);

  for (i = 0; i < n; i++) {
    pid[i] = i + 1;

    printf("Enter arrival time for process[%d]: ", i + 1);
    scanf("%d", &at[i]);

    printf("Enter burst time for process[%d]: ", i + 1);
    scanf("%d", &bt[i]);
  }

  // sorting the processes by AT
  for (i = 0; i < n; i++) {
    for (j = 0; j < n - i - 1; j++) {
      if (at[j] > at[j + 1]) {
        // swapping PID, AT, and BT
        temp = pid[j];
        pid[j] = pid[j + 1];
        pid[j + 1] = temp;
        temp = at[j];
        at[j] = at[j + 1];
        at[j + 1] = temp;
        temp = bt[j];
        bt[j] = bt[j + 1];
        bt[j + 1] = temp;
      }
    }
  }

  // CT
  for (i = 0; i < n; i++) {
    // CPU idle time
    if (curr_time < at[i]) {
      curr_time = at[i];
    }

    // calculating start time for RT
```

```
    st[i] = curr_time;

    curr_time += bt[i];
    ct[i] = curr_time;
  }

  // TAT
  for (i = 0; i < n; i++) {
    tat[i] = ct[i] - at[i];
    tat_sum += tat[i];
  }

  // WT
  for (i = 0; i < n; i++) {
    wt[i] = tat[i] - bt[i];
    wt_sum += wt[i];
  }

  // RT
  for (i = 0; i < n; i++) {
    rt[i] = st[i] - at[i];
  }

  // header row
  printf("\nP\tAT\tBT\tCT\tTAT\tWT\tRT\n\n");

  // sorted by PID
  for (j = 1; j < n + 1; j++) {
    for (i = 0; i < n; i++) {
      if (pid[i] == j)
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n", pid[i], at[i], bt[i], ct[i],
               tat[i], wt[i], rt[i]);
    }
  }

  printf("\nAverage TAT: %.2f\n", (tat_sum / n));
  printf("Average WT: %.2f\n", (wt_sum / n));

  return 0;
}
```

Output:

```
Name: ROHAN PODDAR
USN:1WA24CS238

Enter no. of processes: 4
Enter arrival time for process[1]: 0
Enter burst time for process[1]: 7
Enter arrival time for process[2]: 8
Enter burst time for process[2]: 3
Enter arrival time for process[3]: 3
Enter burst time for process[3]: 4
Enter arrival time for process[4]: 5
Enter burst time for process[4]: 6

P       AT      BT      CT      TAT     WT      RT

P1      0       7       7       7       0       0
P2      8       3       20      12      9       9
P3      3       4       11      8       4       4
P4      5       6       17      12      6       6

Average TAT: 9.75
Average WT: 4.75
```

SJF

Program:

```
#include <stdio.h>

int main() {
  int n, curr_time = 0, pid[10], at[10], bt[10], ct[10], tat[10], wt[10],
        st[10], rt[10], i, j, temp, completed = 0, done[10] = {0};
  double tat_sum = 0, wt_sum = 0;

  printf("Enter no. of processes: ");
  scanf("%d", &n);

  for (i = 0; i < n; i++) {
    pid[i] = i + 1;

    printf("Enter arrival time for process[%d]: ", i + 1);
    scanf("%d", &at[i]);

    printf("Enter burst time for process[%d]: ", i + 1);
    scanf("%d", &bt[i]);
  }

  // sorting the processes by AT
  for (i = 0; i < n; i++) {
    for (j = 0; j < n - i - 1; j++) {
      if (at[j] > at[j + 1]) {
        // swapping PID, AT, and BT
        temp = pid[j];
        pid[j] = pid[j + 1];
        pid[j + 1] = temp;
        temp = at[j];
        at[j] = at[j + 1];
        at[j + 1] = temp;
        temp = bt[j];
        bt[j] = bt[j + 1];
        bt[j + 1] = temp;
      }
    }
  }

 while (completed != n) {
    int min = 99999999, // to set the lowest remaining time
        s = -1;         // index of the process with the shortest remaining time

    for (i = 0; i < n; i++) {
      // 1. AT is now or before current time
      // 2. check if the process is not completed
      // 3. least burst time
```

```
    if (at[i] <= curr_time && !done[i] && bt[i] < min) {
      min = bt[i];
      s = i;
    }
  }

  // CPU idle time
  if (s == -1) {
      curr_time++;
      continue;
  }

  // set the start time
  st[s] = curr_time;

  curr_time += bt[s];

    // CT
    ct[s] = curr_time;

    // TAT
    tat[s] = ct[s] - at[s];
    tat_sum += tat[s];

    // WT
    wt[s] = tat[s] - bt[s];
    wt_sum += wt[s];

    // RT
    rt[s] = st[s] - at[s];

    // increment the completed processes count
    completed++;
    done[s] = 1;
}


// header row
printf("\nP\tAT\tBT\tCT\tTAT\tWT\tRT\n\n");

// sorted by PID
for (j = 1; j < n + 1; j++) {
  for (i = 0; i < n; i++) {
    if (pid[i] == j)
      printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n", pid[i], at[i], bt[i], ct[i],
             tat[i], wt[i], rt[i]);
  }
}
```

```
  printf("\nAverage TAT: %.2f\n", (tat_sum / n));
  printf("Average WT: %.2f\n", (wt_sum / n));

  return 0;
}
```

Output:

```
Name: ROHAN PODDAR
USN:1WA24CS238

Enter no. of processes: 4
Enter arrival time for process[1]: 0
Enter burst time for process[1]: 7
Enter arrival time for process[2]: 8
Enter burst time for process[2]: 3
Enter arrival time for process[3]: 3
Enter burst time for process[3]: 4
Enter arrival time for process[4]: 5
Enter burst time for process[4]: 6

P       AT      BT      CT      TAT     WT      RT

P1      0       7       7       7       0       0
P2      8       3       14      6       3       3
P3      3       4       11      8       4       4
P4      5       6       20      15      9       9

Average TAT: 9.00
Average WT: 4.00
```

SRTF

Program:

```
#include <stdio.h>

int main() {
  int n, completed = 0, curr_time = 0, pid[10], at[10], bt[10], remt[10],
        ct[10], tat[10], wt[10], st[10], rt[10], i, j;
  double tat_sum = 0, wt_sum = 0;

  printf("Enter no. of processes: ");
  scanf("%d", &n);

  for (i = 0; i < n; i++) {
    pid[i] = i + 1;

    printf("Enter arrival time for process[%d]: ", i + 1);
    scanf("%d", &at[i]);

    printf("Enter burst time for process[%d]: ", i + 1);
    scanf("%d", &bt[i]);
    remt[i] = bt[i]; // initialising remaining time
  }

  // running a while loop for each time interval
  // until all the processes have completed
  while (completed != n) {

    int min = 99999999, // to set the lowest remaining time
        s = -1;         // index of the process with the shortest remaining time

    for (i = 0; i < n; i++) {
      // 1. AT is now or before current time
      // 2. process has some remaining time
      // 3. least remaining time
      if (at[i] <= curr_time && remt[i] > 0 && remt[i] < min) {
        min = remt[i];
        s = i;
      }
    }

    // CPU idle time
    if (s == -1) {
      curr_time++;
      continue;
    }

    // if the process is in the CPU for the first time
```

```
    if (remt[s] == bt[s])
      st[s] = curr_time;

    // decrement the remaining time of the process
    remt[s]--;

    // if the process has completed
    if (remt[s] == 0) {
      // CT
      ct[s] = curr_time + 1;

      // TAT
      tat[s] = ct[s] - at[s];
      tat_sum += tat[s];

      // WT
      wt[s] = tat[s] - bt[s];
      wt_sum += wt[s];

      // RT
      rt[s] = st[s] - at[s];

      // increment the completed processes count
      completed++;
    }
    curr_time++;
  }

  // header row
  printf("\nP\tAT\tBT\tCT\tTAT\tWT\tRT\n\n");

  // sorted by PID
  for (j = 1; j < n + 1; j++) {
    for (i = 0; i < n; i++) {
      if (pid[i] == j)
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n", pid[i], at[i], bt[i], ct[i],
               tat[i], wt[i], rt[i]);
    }
  }

  printf("\nAverage TAT: %.2f\n", (tat_sum / n));
  printf("Average WT: %.2f\n", (wt_sum / n));
}
```

Output:

```
Name: ROHAN PODDAR
USN:1WA24CS238

Enter no. of processes: 4
Enter arrival time for process[1]: 0
Enter burst time for process[1]: 7
Enter arrival time for process[2]: 8
Enter burst time for process[2]: 3
Enter arrival time for process[3]: 3
Enter burst time for process[3]: 4
Enter arrival time for process[4]: 5
Enter burst time for process[4]: 6

P       AT      BT      CT      TAT     WT      RT

P1      0       7       7       7       0       0
P2      8       3       11      3       0       0
P3      3       4       14      11      7       4
P4      5       6       20      15      9       9

Average TAT: 9.00
Average WT: 4.00
```